

PROJECT DESIGN PHASE-II

SOLUTION REQUIREMENTS

Functional & Non-Functional Requirements

Project Name: AI BlogNest API

Team ID: SWTID-2026-6999

Date: 21 September 2026

Maximum Marks: 4 Marks

The provided template requires the proposed solution to be described using Functional Requirements and Non-Functional Requirements.

Functional Requirements

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	User Authentication & Role Management	• User registration through form
		• User login using email and password
		• JWT-based authentication
		• Role-based access control for Admin, Editor, Author and Reader
		• User role assignment and management
FR-2	Blog Content Management	• Create blog posts
		• View blog posts
		• Update blog posts
		• Delete blog posts
		• Manage draft, pending, scheduled and published states
		• Manage categories and tags
FR-3	AI-Powered Content Services	• Generate structured blog title, content, summary and tags
		• Summarize existing blog content using AI
		• Use prompt engineering for structured AI responses

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-4	Search, Comments & Content Discovery	Keyword-based blog search using MongoDB Atlas Search
		Semantic search using vector embeddings and MongoDB Atlas Vector Search
		Add comments to published blogs
		View comments
		Edit and delete comments according to permissions
		Moderate comments using pending, approved and spam status

The functional requirements are based on the actual BlogNest features: authentication, role management, blog management, categories/tags, comments, AI generation/summarization, and search capabilities.

Non-Functional Requirements

NFR No.	Non-Functional Requirement	Description
NFR-1	Usability	The REST APIs should provide clear request formats, consistent responses and meaningful error messages so that users and developers can interact with the system easily.
NFR-2	Security	The system should protect user accounts and API resources using bcrypt password hashing, JWT authentication, role-based authorization, input validation, sanitization and rate limiting.
NFR-3	Reliability	The system should handle invalid requests and service failures gracefully while maintaining data consistency and returning appropriate responses.
NFR-4	Performance	API requests, database operations, keyword searches and semantic searches should be processed efficiently with suitable database indexing and optimized backend operations.
NFR-5	Availability	The backend should remain accessible whenever the Node.js server, MongoDB Atlas and required AI services are available.
NFR-6	Scalability	The modular MVC and service-based architecture should allow additional users, blogs, AI features and search operations to be added without major changes to the existing system.